

CloudPulse

Updated Milestone Document — Milestone 3

Project: Cloud Service Health Monitoring and Alert Dashboard
Team: Katha Patel • Ashwin S. Thankachan • Meet Patel • Pavithra Prasad
Repository: github.com/KathaPatel29/csye6300-summer26-group3-final-project

1. Project Group Number and Name

Group number: 3
Group name: CloudPulse Team

- Katha Patel
- Ashwin S Thankachan
- Meet Patel
- Pavithra Prasad

2. Project Topic and Name

CloudPulse — Cloud Service Health Monitoring and Alert Dashboard

CloudPulse is a student-scale observability application that registers services, runs scheduled or manual health checks, normalizes external response formats, evaluates health, records lifecycle transitions, raises alerts, and presents current service state through a React dashboard.

3. Technology Stack

Layer	Technologies	Milestone 3 use
Backend	Java 22; Spring Boot 3.3.4; Spring Web; Scheduling; Data JPA; Validation	REST API, monitoring workflow, scheduling, domain rules, persistence
Frontend	React 19; TypeScript 5.7; Vite 6; Tailwind CSS 4	Service dashboard, summary cards, registration, edit/delete UI
Data	H2; PostgreSQL 16	H2 development/testing; PostgreSQL Docker profile
Infrastructure	Docker Compose; backend Dockerfile	Compose configuration for backend and PostgreSQL startup
Testing	JUnit 5; Mockito; Spring Boot Test; MockMvc	Unit, controller, scheduler, workflow, and Observer integration tests
Documentation/CI	OpenAPI/Swagger; GitHub Actions; Maven/npm	API discovery and backend/frontend verification

4. Functionalities Implemented for Milestone 3

Area	Verified Milestone 3 progress	Repository evidence
React dashboard	Service list, status badges, system summary cards, loading/error/empty states, responsive Tailwind UI.	frontend/src/App.tsx and components/
Service CRUD	Register, edit, and delete services from the UI; backend PUT/DELETE endpoints; deletion blocked with HTTP 409 when history exists.	ServiceForm, ServiceList, ServiceController
History and policy APIs	Read health-check history and status-event history; change a service's NORMAL/STRICT evaluation policy.	Three dedicated REST controllers and tests
Live Observer flow	Spring registers all subscribers; committed status transitions publish events; one subscriber failure does not stop others.	ObserverRegistrar, StatusChangeHandler, integration test
Scheduling/decorators	Per-service check intervals, rescheduling/cancellation, scheduler pool, and live MetricsMonitorDecorator wiring.	HealthCheckScheduler and SchedulerConfig
Deployment	Dockerfile and Docker Compose definitions for Spring Boot plus PostgreSQL 16 with configured health-gated startup.	backend/Dockerfile and docker-compose.yml
API hardening	Consistent JSON errors for validation, 404, 405, and service-history conflicts; alert 404 behavior aligned.	GlobalExceptionHandler and controller tests
Verification	Latest packaged Surefire reports contain 77 tests with 0 failures and 0 errors; compiled frontend dist is present.	backend/target/surefire-reports; frontend/dist

5. Design Patterns Implemented — Code Snippets

5.1 Factory Method

Problem solved: Selects HTTP or mock monitors without coupling callers to concrete construction.

Current participants: MonitorCreator, HttpMonitorCreator, MockMonitorCreator, ServiceMonitor

Factory Method excerpt — *backend/.../monitor/MonitorCreator.java + backend/.../scheduler/HealthCheckScheduler.java*

```
public abstract class MonitorCreator {
    public abstract ServiceMonitor createMonitor();
}

private MonitorCreator resolveCreator(MonitorType monitorType) {
    return switch (monitorType) {
        case HTTP -> httpMonitorCreator;
        case MOCK -> mockMonitorCreator;
    };
}
```

5.2 Adapter

Problem solved: Normalizes standard and legacy health-response bodies into one HealthCheckResult model.

Current participants: HealthResponseAdapter, StandardHealthAdapter, LegacyHealthAdapter

Adapter excerpt — *backend/.../adapter/HealthResponseAdapter.java*

```
public interface HealthResponseAdapter {
    HealthCheckResult adapt(
        String rawResponse, int httpStatus, long responseTimeMs);
    boolean supports(String rawResponse);
}
```

```
// StandardHealthAdapter and LegacyHealthAdapter both
// return the same HealthCheckResult domain type.
```

5.3 Decorator

Problem solved: Layers logging, retry, and metrics behavior around a monitor without changing it.

Current participants: MonitorDecorator, LoggingMonitorDecorator, RetryMonitorDecorator, MetricsMonitorDecorator

Decorator excerpt — *backend/.../scheduler/HealthCheckScheduler.java*

```
private ServiceMonitor createDecoratedMonitor(
    MonitoredService service) {
    MonitorCreator creator = resolveCreator(service.getMonitorType());
    ServiceMonitor baseMonitor = creator.createMonitor();
    return new MetricsMonitorDecorator(
        new RetryMonitorDecorator(
            new LoggingMonitorDecorator(baseMonitor)));
}
```

5.4 Strategy

Problem solved: Allows NORMAL and STRICT latency thresholds to vary by monitored service.

Current participants: HealthEvaluationStrategy, NormalHealthStrategy, StrictHealthStrategy, HealthEvaluationService

Strategy excerpt — *backend/.../evaluation/NormalHealthStrategy.java*

```
public ServiceStatus evaluate(HealthCheckSnapshot snapshot) {
    if (!snapshot.reachable() || snapshot.httpStatus() >= 400) {
        return ServiceStatus.DOWN;
    }
    if (snapshot.responseTimeMs() > 1_000) {
        return ServiceStatus.DEGRADED;
    }
    return ServiceStatus.HEALTHY;
}
```

5.5 State

Problem solved: Controls lifecycle transitions and explicit recovery according to the current service state.

Current participants: ServiceState, ServiceStateMachine, Unknown/Healthy/Degraded/Down/Recovered states

State excerpt — *backend/.../lifecycle/DownState.java*

```
public class DownState implements ServiceState {
    public ServiceStatus status() { return ServiceStatus.DOWN; }

    public ServiceStatus transition(ServiceStatus observedStatus) {
        return observedStatus == ServiceStatus.HEALTHY
            ? ServiceStatus.RECOVERED
            : observedStatus;
    }
}
```

5.6 Chain of Responsibility

Problem solved: Processes validation, availability, latency, status changes, and alerts as ordered independent stages.

Current participants: ResultHandler and five concrete handlers, HealthCheckProcessor

Chain of Responsibility excerpt — *backend/.../handler/HealthCheckProcessor.java*

```
ResultHandler validation = new ResponseValidationHandler();
validation.setNext(new AvailabilityHandler())
    .setNext(new LatencyHandler(evaluationService))
    .setNext(new StatusChangeHandler(
        statusEventRepository, monitoredServiceRepository,
        stateMachine, statusEventPublisher))
    .setNext(new AlertCreationHandler(alertRepository));
this.chainHead = validation;
```

5.7 Command

Problem solved: Encapsulates checks, monitoring toggles, alert acknowledgement, and resolution behind consistent execution boundaries.

Current participants: MonitoringCommand and five concrete commands

Command excerpt — *backend/.../command/RunHealthCheckCommand.java*

```
public interface MonitoringCommand {
    void execute();
}

public void execute() {
    HealthCheckResult result = monitor.check(service);
    result.setService(service);
    processor.process(result);
    resultRepository.save(result);
    service.setLastCheckedAt(Instant.now());
    serviceRepository.save(service);
}
```

5.8 Observer

Problem solved: Publishes committed status events to isolated dashboard, alert, and history subscribers.

Current participants: StatusEventPublisher, ObserverRegistrar, EventSubscriber implementations

Observer excerpt — *backend/.../observer/ObserverRegistrar.java*

```
@PostConstruct
void registerObservers() {
    publisher.register(dashboardUpdateObserver);
    publisher.register(alertObserver);
    publisher.register(eventHistoryObserver);
}

// StatusChangeHandler calls publisher after commit;
// StatusEventPublisher isolates failures per subscriber.
```

5.9 MVC

Problem solved: Separates the React View, Spring REST controllers, domain/services, and repositories.

Current participants: React components, ServiceController, AlertController, domain model, repositories

MVC excerpt — *frontend/src/api/services.ts + ServiceController.java*

```
export function updateService(id: string, request: RegisterServiceRequest) {
    return put<MonitoredService>(`/services/${id}`, request)
}

@PutMapping("/{id}")
public MonitoredService updateService(
    @PathVariable UUID id,
    @Valid @RequestBody RegisterServiceRequest request) {
    MonitoredService service = findServiceOrThrow(id);
    service.setName(request.name());
    service.setHealthUrl(request.healthUrl());
    service.setMonitorType(request.monitorType());
    return serviceRepository.save(service);
}
```

6. Current System Demonstration

1. Start PostgreSQL and the backend with docker compose up --build, or run the backend with its H2 development profile.
2. Start the Standard, Legacy, and Flaky demo services on ports 8081, 8082, and 8083.
3. Start the Vite frontend and register services from the dashboard.
4. Observe per-service scheduled checks, status badges, summary counts, persisted results, lifecycle events, and generated alerts.
5. Use Swagger for manual checks, monitoring toggles, alert actions, evaluation-policy changes, and history endpoints that are not yet exposed in the dashboard.
6. Edit a service before it has history, and demonstrate the protected delete behavior after monitoring history exists.

7. Functionalities Planned for Final Submission

- Complete React alert views with acknowledgement and resolution controls.
- Expose manual health check, enable/disable monitoring, and evaluation-policy selection in the dashboard.
- Add health-check history and status-event timeline views using the implemented read APIs.
- Add frontend component tests and browser-level end-to-end coverage for the main operator workflows.
- Expand Docker Compose to include the frontend and three demo services, then validate the complete PostgreSQL-backed demonstration.
- Add stable seed/demo data, cross-layer error handling, accessibility checks, and final UI polish.
- Update the README and all milestone/design artifacts so setup commands and implementation status match the final repository.
- Run final backend, frontend, Docker, and demonstration verification in CI and record the results.

8. Team Contributions

Member	Milestone 3 contribution
Katha Patel	Implemented per-service scheduling and rescheduling, wired the metrics decorator, added the backend Dockerfile and PostgreSQL Docker Compose configuration, and corrected scheduler configuration.
Ashwin S Thankachan	Added evaluation-policy and health-check-history APIs with DTOs and controller tests, and supported documentation/integration review.
Meet Patel	Connected status-event publication to the processing workflow, added the status-event history endpoint and API error handling, and standardized AlertController 404 responses.
Pavithra Prasad	Built the React service dashboard and CRUD UI, added backend service update/delete behavior, completed live Observer registration/isolation and integration coverage, and fixed controller-test isolation.